We will now begin the **implementation-ready REST API reference**. The following structure is written as if it will be converted directly into an **OpenAPI 3.1 specification**, SDK documentation, and backend implementation guide.

TRADIE MASTER DOCUMENTATION

Volume 08 – Enterprise API Reference, SDKs & Integration Handbook

Installment 2 – Authentication, Organization, User Management & Marketplace REST APIs

Estimated Final Size: 800–1,000 pages

Purpose

This installment defines the first set of production-ready REST APIs for identity, organization management, user administration, producer onboarding, marketplace, RFQs, auctions, and trade creation.

PART II

Authentication APIs

API 001 – User Login

Endpoint

POST /api/v1/auth/login

Purpose

Authenticates a user and returns access and refresh tokens.

Authentication

None (public endpoint)

Request Body

```
{
  "username": "user@example.com",
  "password": "********",
```

```
  "deviceId": "DEVICE-12345",
  "clientVersion": "1.0.0"
}
```

Success Response (200)

```
{
  "success": true,
  "data": {
    "accessToken": "<JWT>",
    "refreshToken": "<JWT>",
    "expiresIn": 3600,
    "user": {
      "userId": 1001,
      "displayName": "Ramesh Kumar",
      "organizationId": 501,
      "roles": [
        "PRODUCER"
      ]
    }
  }
}
```

Error Codes

Code	Description
AUTH-001	Invalid credentials
AUTH-002	Account disabled
AUTH-003	Password expired
AUTH-004	MFA required
AUTH-005	Account locked

Business Rules

- Username must exist.
- Password policy enforced.
- Failed attempts contribute to account lockout.

- MFA challenge issued when configured.

Events

- UserLoggedIn
- LoginFailed
- MFARequested

API 002 – Refresh Token

Endpoint

POST /api/v1/auth/refresh

Purpose

Issue a new access token using a valid refresh token.

Authentication

Refresh Token

Response

- New access token.
- Updated expiry.
- Session validation.

API 003 – Logout

Endpoint

POST /api/v1/auth/logout

Actions

- Invalidate refresh token.
- Close current session.
- Audit log entry.
- Publish logout event.

API 004 – Forgot Password

POST /api/v1/auth/password/forgot

Workflow

User
↓
OTP Generated
↓
Email/SMS
↓
OTP Verification
↓
Password Reset

PART III

Organization APIs

API 010 – Create Organization

POST /api/v1/organizations

Authorization

System Administrator

Registration Workflow

Request

```
{
 "organizationName": "ABC Traders",
 "legalName": "ABC Traders Pvt Ltd",
 "country": "India",
 "state": "Andhra Pradesh",
 "district": "Guntur",
 "businessType": "Commission Agent"
}
```

Validation

- Organization name uniqueness.
- Legal registration verification.
- Mandatory address.
- Business type supported.

Response

```
{
 "organizationId": 501,
 "status": "PENDING_VERIFICATION"
}
```

API 011 – Get Organization

GET /api/v1/organizations/{organizationId}

Supports:

- Summary view.
- Detailed profile.
- Related entities.
- Audit information.

API 012 – Update Organization

PATCH /api/v1/organizations/{organizationId}

Supports partial updates with optimistic concurrency control using entity versioning.

PART IV

User Management APIs

API 020 – Create User

POST /api/v1/users

Request

```
{
 "firstName": "Ramesh",
 "lastName": "Kumar",
 "email": "ramesh@example.com",
 "mobile": "+919999999999",
 "roleIds": [12]
}
```

Validation

- Email uniqueness.
- Mobile uniqueness (where configured).
- Role authorization.
- License availability.

API 021 – Search Users

GET /api/v1/users

Supports

- Pagination.
- Sorting.
- Full-text search.
- Filters by role, status, organization, last login.

API 022 – Assign Role

POST /api/v1/users/{userId}/roles

Business Rules

- Requester must possess role assignment privileges.
- Role conflicts validated.

- Separation of duties enforced.

PART V

Producer APIs

API 030 – Register Farm

POST /api/v1/farms

Fields

- Farm name.
- GPS coordinates.
- Survey number.
- Total area.
- Irrigation type.
- Certifications.
- Owner details.

Events

- FarmRegistered
- FarmVerified

API 031 – Register Commodity

POST /api/v1/commodities/register

Validation

- Commodity exists.
- Variety valid.
- Harvest season valid.
- Duplicate prevention.

API 032 – Publish Listing

POST /api/v1/marketplace/listings

Listing Types

- Immediate Sale.
- Auction.
- Negotiation.
- RFQ Response.
- Contract Listing.

Business Rules

- Quantity > 0.
- Price policy validation.
- Required quality information.
- Photos optional/configurable.

PART VI

Buyer APIs

API 040 – Search Marketplace

GET /api/v1/marketplace/listings

Filters

- Commodity.
- Variety.
- Grade.
- Warehouse.
- Price.

- Distance.
- Seller rating.
- Certification.

API 041 – Create RFQ

POST /api/v1/rfqs

Request

```
{
 "commodityId": 18,
 "quantity": 100000,
 "deliveryDate": "2026-08-15",
 "budget": 17250
}
```

Events

- RFQCreated
- RFQPublished

PART VII

Auction APIs

API 050 – Create Auction

POST /api/v1/auctions

Validation

- Listing exists.
- Reserve price policy.
- Auction duration.
- Participant rules.

API 051 – Place Bid

POST /api/v1/auctions/{auctionId}/bids

Business Rules

- Auction active.
- Bid exceeds current highest bid by minimum increment.
- Bidder eligible.
- Sufficient credit (if applicable).

Events

- BidPlaced
- HighestBidUpdated
- OutBidNotification

PART VIII

Trade APIs

API 060 – Create Trade

POST /api/v1/trades

Request

```
{
  "buyerId": 501,
  "sellerId": 220,
  "commodityId": 18,
  "quantity": 25000,
  "unitPrice": 17250
}
```

Validation

- Buyer active.

- Seller active.
- Commodity available.
- Inventory sufficient (where applicable).
- Quantity within configured limits.
- Credit checks completed (if enabled).

Response


```
{
 "tradeId": 800011,
 "status": "NEGOTIATION"
}
```


Events

- TradeCreated
- TradeValidated
- TradeNegotiationStarted

Common API Standards

Every endpoint documents:

- Purpose.
- URL.
- HTTP method.
- Authentication.
- Authorization scopes.
- Headers.
- Request schema.
- Response schema.
- Validation rules.
- Business rules.

- Error codes.
- Audit events.
- Domain events.
- Idempotency requirements.
- Rate limits.
- Example requests.
- Example responses.
- SDK examples.
- OpenAPI schema references.

End of Installment 2

This installment establishes the production-ready REST API specifications for authentication, organization management, user administration, producer operations, buyer workflows, marketplace listings, RFQs, auctions, and trade creation.

Preview – Installment 3: Warehouse, Inventory, Logistics, Laboratory & Finance APIs (≈900–1,100 pages)

The next installment will define:

- Warehouse receiving APIs.
- Weighbridge integration APIs.
- Inventory APIs.
- Bin and location APIs.
- Dispatch APIs.
- Shipment APIs.
- Fleet and driver APIs.
- Laboratory sample APIs.
- Quality inspection APIs.

- Certificate APIs.
- Accounts Receivable APIs.
- Accounts Payable APIs.
- General Ledger APIs.
- Banking APIs.
- Settlement APIs.
- Tax APIs.
- Reporting APIs.
- Webhook specifications.
- OpenAPI 3.1 schemas.
- SDK implementations (Flutter, React, Java, .NET, Python, Go).
- Event payloads and sequence diagrams.

This installment will complete the operational and financial API layer, enabling end-to-end backend implementation across the Tradie platform.

We will now complete the **core operational backend APIs** that power warehouse management, inventory control, logistics, laboratory workflows, finance, taxation, settlements, and reporting.

TRADIE MASTER DOCUMENTATION

Volume 08 – Enterprise API Reference, SDKs & Integration Handbook

Installment 3 – Warehouse, Inventory, Logistics, Laboratory & Finance APIs

Estimated Final Size: 1,000–1,300 pages

Purpose

This installment provides production-ready REST API specifications for warehouse operations, inventory management, logistics execution, laboratory quality workflows, finance, settlements, taxation, and reporting.

PART IX

Warehouse APIs

API 070 – Create Warehouse Receipt

Endpoint

POST /api/v1/warehouse/receipts

Purpose

Register the receipt of commodities into the warehouse after successful verification, weighment, and quality checks.

Authentication

Warehouse Operator, Warehouse Manager

Request Body

```
{
 "tradeId": 800011,
 "warehouseId": 21,
 "vehicleNumber": "AP39AB1234",
 "commodityId": 18,
 "grossWeight": 25250,
 "tareWeight": 1250,
 "samplingCompleted": true
}
```

Business Rules

- Trade must exist.
- Warehouse must be active.
- Gross weight must exceed tare weight.
- Sampling required when configured.
- Capacity validation before receipt creation.

Events

- WarehouseReceiptCreated

- InventoryReserved
- AuditLogGenerated

API 071 – Allocate Storage Bin

POST /api/v1/warehouse/storage-allocation

Allocation Factors

- Commodity
- Grade
- Moisture
- Temperature requirement
- Bin availability
- FIFO/FEFO policy

Response


```
{
 "allocationId": 5501,
 "binCode": "A-12-07",
 "status": "ALLOCATED"
}
```

API 072 – Dispatch Commodity

POST /api/v1/warehouse/dispatch

Workflow

Dispatch Request
↓
Inventory Validation
↓
Loading
↓
Shipment Created

↓

Inventory Updated

Validation

- Inventory available.
- Shipment scheduled.
- Dispatch authorization.
- Vehicle assigned.

PART X

Inventory APIs

API 080 – Search Inventory

GET /api/v1/inventory

Filters

- Commodity
- Grade
- Warehouse
- Lot
- Quality Status
- Reserved
- Available

Response Fields

- Lot Number
- Warehouse
- Quantity
- Reserved Quantity

- Available Quantity
- Age
- Expiry
- Storage Location

API 081 – Stock Movement

POST /api/v1/inventory/movements

Movement Types

- Receipt
- Transfer
- Reservation
- Adjustment
- Dispatch
- Return
- Write-off

Business Rules

- Inventory cannot become negative.
- All movements require audit entries.
- High-value adjustments require approval.

API 082 – Cycle Count

POST /api/v1/inventory/cycle-count

Request


```
{
  "warehouseId": 21,
  "location": "Zone-A",
```

"countType": "FULL"
}

Events

- CycleCountStarted
- VarianceDetected
- CountApproved

PART XI

Logistics APIs

API 090 – Create Shipment

POST /api/v1/logistics/shipments

Fields

- Dispatch ID
- Vehicle
- Driver
- Route
- Expected Delivery
- GPS Enabled

Events

- ShipmentCreated
- RouteAssigned

API 091 – Track Shipment

GET /api/v1/logistics/shipments/{shipmentId}

Returns

- Current Location
- ETA
- Route Progress
- Stops
- Temperature
- Alerts

API 092 – Proof of Delivery

POST /api/v1/logistics/proof-of-delivery

Uploads

- Signature
- Images
- GPS Coordinates
- Timestamp

Events

- DeliveryCompleted
- InventoryDelivered

PART XII

Laboratory APIs

API 100 – Register Sample

POST /api/v1/laboratory/samples

Fields

- Commodity
- Lot

- Warehouse
- Sampling Method
- Collector
- Seal Number

Events

- SampleCollected
- SampleRegistered

API 101 – Submit Test Results

POST /api/v1/laboratory/tests

Parameters

- Moisture
- Foreign Matter
- Colour
- Size
- Pungency
- Aflatoxin
- Remarks

Validation

- Test template applicable.
- Mandatory parameters completed.
- Analyst authorization.

API 102 – Generate Certificate

POST /api/v1/laboratory/certificates

Response

{
"certificateId": 8701,
"certificateNumber": "QC-2026-000871",
"status": "ISSUED"
}

Events

- CertificateIssued
- CertificatePublished

PART XIII

Finance APIs

API 110 – Generate Invoice

POST /api/v1/finance/invoices

Business Rules

- Trade completed.
- Pricing finalized.
- Tax rules applied.
- Duplicate prevention.

Response

{
"invoiceId": 6101,
"invoiceNumber": "INV-2026-100021"
}

API 111 – Record Payment

POST /api/v1/finance/payments

Fields

- Invoice
- Amount
- Bank
- Reference Number
- Payment Mode
- Payment Date

Events

- PaymentReceived
- LedgerUpdated

API 112 – Journal Entry

POST /api/v1/finance/general-ledger/journals

Validation

- Balanced entries.
- Open accounting period.
- Authorized user.
- Valid chart of accounts.

API 113 – Settlement

POST /api/v1/finance/settlements

Workflow

Trade
↓
Invoice
↓
Payment
↓
Settlement

↓

Ledger Posting

PART XIV

Taxation APIs

API 120 – Tax Calculation

POST /api/v1/tax/calculate

Inputs

- Commodity
- Jurisdiction
- Buyer
- Seller
- Amount

Output

- Applicable taxes
- Exemptions
- Final payable amount

API 121 – Generate Tax Return

POST /api/v1/tax/returns

Features

- Period selection
- Validation
- Return generation
- Filing status

PART XV

Reporting APIs

API 130 – Dashboard Reports

GET /api/v1/reports/dashboard

Supports:

- Role-specific dashboards.
- KPI summaries.
- Time filters.
- Export options.

API 131 – Financial Reports

GET /api/v1/reports/finance

Supported Reports

- Trial Balance
- Profit & Loss
- Balance Sheet
- Cash Flow
- Receivables Aging
- Payables Aging

API 132 – Operational Reports

GET /api/v1/reports/operations

Supported Reports

- Warehouse Occupancy

- Inventory Aging
- Shipment Performance
- Laboratory Turnaround
- Auction Performance
- Trade Volume

Webhook Specifications

Tradie supports outbound event notifications for external systems.

Example Webhook

POST https://partner.example.com/webhooks/trade

Payload

```
{
 "event": "TradeCompleted",
 "tradeId": 800011,
 "timestamp": "2026-06-27T14:30:00Z"
}
```

Delivery Rules

- Signed payloads (HMAC).
- Retry with exponential backoff.
- Dead-letter queue after configurable retries.
- Idempotent event identifiers.

SDK Support Matrix

Official SDKs will be published for:

Language / Platform	Support
Java	✅

Language / Platform Support

Kotlin	✅
.NET	✅
Python	✅
Go	✅
Node.js	✅
TypeScript	✅
Flutter (Dart)	✅
React	✅
React Native	✅
Android (Native)	✅
Swift (iOS)	✅

Each SDK includes:

- Authentication helpers.
- Typed models.
- Error handling.
- Retry policies.
- Pagination utilities.
- WebSocket support.
- Event subscription helpers.
- Logging hooks.
- Sample applications.

End of Installment 3

This installment defines the production-ready REST APIs for warehouse management, inventory operations, logistics, laboratory workflows, finance, taxation, reporting, and external webhook integration, completing the core operational API layer.

Preview – Installment 4: GraphQL, gRPC, WebSockets, Kafka Events, SDK Architecture & Integration Patterns (≈1,200–1,500 pages)

The next installment will define:

- Complete GraphQL schema and resolvers.
- gRPC service contracts with Protocol Buffers.
- Real-time WebSocket channels for auctions, notifications, shipment tracking, and dashboards.
- Kafka topic catalog and versioned event schemas.
- Event sourcing and CQRS patterns.
- Saga orchestration and distributed transaction flows.
- Integration patterns for ERP, banking, payment gateways, IoT devices, laboratory instruments, and government portals.
- SDK architecture and code generation strategy.
- OpenAPI and AsyncAPI governance.
- End-to-end sequence diagrams.
- Performance tuning, resilience patterns, circuit breakers, retries, and observability standards.

This installment will complete the enterprise integration architecture, providing a comprehensive reference for building scalable, event-driven, production-grade integrations across the entire Tradie ecosystem.

We now begin the **advanced integration layer**, which defines how Tradie's services communicate internally and externally using synchronous and asynchronous technologies. This installment provides the architectural contracts required for high-scale, event-driven enterprise deployments.

TRADIE MASTER DOCUMENTATION

Volume 08 – Enterprise API Reference, SDKs & Integration Handbook

Installment 4 – GraphQL, gRPC, WebSockets, Event Streaming, SDK Architecture & Enterprise Integration Patterns

Estimated Final Size: 1,500–1,800 pages

Audience

- Enterprise Architects
- Backend Engineers
- Integration Teams
- Platform Engineers
- DevOps Engineers
- SDK Developers
- Performance Engineers

PART XVI

GraphQL Architecture

Chapter 271

GraphQL Philosophy

GraphQL is provided alongside REST to support:

- Mobile applications
- Complex dashboards
- Low-bandwidth environments
- Aggregated enterprise data
- Reduced API round-trips

- Client-driven data selection

REST remains the authoritative command interface, while GraphQL is optimized for flexible querying and dashboard composition.

GraphQL Endpoint

POST /graphql

Authentication

- OAuth2 Bearer Token
- JWT
- API Key (configured integrations only)

Root Schema

```
type Query {
  me: User
  organization(id: ID!): Organization
  commodity(id: ID!): Commodity
  marketplace(filter: MarketplaceFilter): [Listing]
  warehouse(id: ID!): Warehouse
  inventory(filter: InventoryFilter): [Inventory]
  shipment(id: ID!): Shipment
  dashboard(role: DashboardRole!): Dashboard
}
```

Root Mutations

```
type Mutation {
  login(input: LoginInput!): LoginResponse
  createTrade(input: TradeInput!): Trade
  createRFQ(input: RFQInput!): RFQ
  publishListing(input: ListingInput!): Listing
```

```
  reserveWarehouse(input: ReservationInput!): Reservation
}
```

GraphQL Subscriptions

Supports real-time updates.

```
subscription {
  auctionUpdated(auctionId: ID!)
}
```

Additional subscriptions include:

- ShipmentUpdated
- InventoryChanged
- DashboardMetricsUpdated
- NotificationReceived
- LaboratoryResultPublished

PART XVII

gRPC Services

Service Philosophy

gRPC is recommended for:

- Internal microservice communication
- High-throughput processing
- Low-latency requests
- Streaming operations
- AI services
- Analytics

Example Proto

```
service TradeService {

  rpc CreateTrade(CreateTradeRequest)
     returns (TradeResponse);

  rpc GetTrade(GetTradeRequest)
     returns (TradeResponse);

  rpc SearchTrades(SearchRequest)
     returns (TradeList);
}
```

Streaming Example

```
rpc StreamAuctionUpdates
   (AuctionRequest)
returns (stream AuctionEvent);
```

Core gRPC Services

- TradeService
- MarketplaceService
- CommodityService
- WarehouseService
- InventoryService
- LogisticsService
- FinanceService
- TaxService
- LaboratoryService
- NotificationService
- AnalyticsService

- AIRecommendationService
- AuditService
- IdentityService

PART XVIII

WebSocket Services

Purpose

Support real-time enterprise collaboration and live operational updates.

Endpoint

wss://api.tradie.com/ws

Authentication

JWT during connection establishment.

Supported Channels

Auctions

auction/{auctionId}

Events:

- BidPlaced
- HighestBidChanged
- AuctionPaused
- AuctionClosed

Notifications

notifications/{userId}

Supports:

- Push notifications
- Workflow approvals
- Alerts
- Reminders

Warehouse

warehouse/{warehouseId}

Live updates:

- Receiving
- Dispatch
- Inventory changes
- Capacity alerts

Shipment Tracking

shipment/{shipmentId}

Live telemetry:

- GPS
- Temperature
- ETA
- Delay alerts

PART XIX

Event Streaming

Kafka Topic Catalog

Trading

trade.created.v1

trade.updated.v1

trade.completed.v1

Marketplace

listing.created.v1

listing.updated.v1

listing.closed.v1

Warehouse

warehouse.receipt.created.v1

warehouse.dispatch.completed.v1

inventory.updated.v1

Laboratory

sample.registered.v1

test.completed.v1

certificate.issued.v1

Finance

invoice.generated.v1

payment.received.v1

settlement.completed.v1

Notifications

notification.created.v1

notification.sent.v1

notification.read.v1

Event Envelope

```
{
 "eventId": "UUID",
 "eventType": "TradeCompleted",
 "version": "1.0",
 "timestamp": "2026-06-27T12:30:00Z",
 "correlationId": "ABC-123",
 "tenantId": "TENANT-01",
 "payload": {}
}
```

Event Processing Rules

Every event must support:

- Idempotency
- Ordering (where required)
- Replay
- Dead-letter handling
- Version compatibility

- Schema validation

PART XX

CQRS & Event Sourcing

Command Side

Handles:

- Trade creation
- Settlement
- Inventory updates
- Financial posting

Query Side

Optimized read models for:

- Dashboards
- Analytics
- Search
- Reporting

Event Store

Stores immutable business events.

Examples:

- TradeCreated
- TradeApproved
- WarehouseReceived
- ShipmentDispatched

- InvoiceGenerated
- PaymentReceived

PART XXI

Saga Orchestration

Example: Trade Fulfillment Saga

Trade Created

Inventory Reserved

Warehouse Allocation

Quality Inspection

Invoice Generation

Payment

Dispatch

Settlement Complete

Compensation actions are defined for failures at each step, such as inventory release, reservation cancellation, or invoice reversal.

PART XXII

Enterprise SDK Architecture

Supported SDKs

- Java
- Kotlin
- Python
- Go
- .NET
- Node.js
- TypeScript
- Flutter (Dart)
- React
- React Native
- Swift
- Android Native

SDK Modules

- Authentication
- Organization
- Users
- Marketplace
- Trading
- Warehouse
- Inventory
- Logistics
- Laboratory

- Finance
- Reporting
- Notifications
- AI
- Administration

Common SDK Features

- Automatic token refresh
- Retry with exponential backoff
- Circuit breaker support
- Pagination helpers
- Typed models
- Async operations
- WebSocket client
- GraphQL client
- File upload utilities
- Structured logging
- OpenTelemetry integration

PART XXIII

Enterprise Integration Patterns

Supported Integrations

ERP

- SAP S/4HANA
- Oracle ERP Cloud

- Microsoft Dynamics 365
- Odoo
- ERPNext

Banking

- NEFT
- RTGS
- IMPS
- UPI
- SWIFT

Government

- GST systems
- e-Invoice
- e-Way Bill
- Agricultural market portals

Logistics

- GPS providers
- Fleet management systems
- Courier APIs

Laboratory

- Laboratory Information Management Systems (LIMS)
- Digital laboratory instruments
- Moisture analyzers
- Weighing scales

IoT

- Temperature sensors
- Humidity sensors

- Warehouse environment monitoring
- RFID readers
- Barcode scanners

PART XXIV

Resilience & Reliability

Every integration must support:

- Timeouts
- Retries
- Exponential backoff
- Circuit breakers
- Bulkheads
- Health checks
- Graceful degradation
- Rate limiting
- Fallback mechanisms

Observability

All integration points publish:

- Metrics
- Logs
- Traces
- Business events
- Health status
- Latency measurements
- Error diagnostics

End of Installment 4

This installment defines the advanced integration architecture for Tradie, including GraphQL, gRPC, WebSockets, Kafka-based event streaming, CQRS, Event Sourcing, Saga orchestration, SDK architecture, enterprise integration patterns, and resilience mechanisms.

Preview – Volume 09: Enterprise Data Architecture, Database Design, Security, DevSecOps & Cloud Deployment (≈2,500–3,000 pages)

The next volume will cover the infrastructure and operational foundation of Tradie, including:

- Enterprise domain model and data architecture.
- Database schemas (ERDs) for every module.
- Relational and NoSQL design strategies.
- Master data management.
- Data governance and lifecycle policies.
- Encryption, key management, and secrets handling.
- Identity federation and Zero Trust security architecture.
- DevSecOps pipelines (CI/CD).
- Kubernetes deployment architecture.
- Multi-region disaster recovery.
- Backup and archival strategies.
- Infrastructure as Code (Terraform/Helm).
- Monitoring, logging, and alerting.
- Performance engineering, scalability testing, and production readiness checklists.

This next volume will provide the blueprint required for deploying Tradie as a secure, scalable, enterprise-grade SaaS platform.